

BÀI THỰC HÀNH: DANH SÁCH LIÊN KẾT ĐÔI

1. MỤC TIÊU BÀI THỰC HÀNH

- Hiểu rõ khái niệm và cấu trúc của danh sách liên kết đôi.
- Sử dụng struct và con trỏ để tạo các node và danh sách liên kết đôi.
- Cài đặt các thao tác cơ bản: thêm, xóa, duyệt, tìm kiếm.
- Áp dụng danh sách liên kết đôi vào các bài toán thực tế.

2. CÁC VÍ DỤ MINH HỌA

Ví dụ 1: Tạo và hiển thị danh sách liên kết đôi

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
    Node(int value) {
        data = value;
        prev = nullptr;
        next = nullptr;
    }
};

void insertAtHead(Node*& head, int value) {
    Node* newNode = new Node(value);
    if (head != nullptr) {
        newNode->next = head;
        head->prev = newNode;
    }
    head = newNode;
}

void displayForward(Node* head) {
    cout << "Danh sách (từ đầu đến cuối): ";
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " ";
    }
}
```

```

        temp = temp->next;
    }
    cout << endl;
}

void displayBackward(Node* head) {
    if (head == nullptr) return;
    Node* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
    cout << "Danh sách (từ cuối về đầu): ";
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->prev;
    }
    cout << endl;
}

int main() {
    Node* head = nullptr;
    insertAtHead(head, 10);
    insertAtHead(head, 20);
    insertAtHead(head, 30);
    displayForward(head);
    displayBackward(head);
    return 0;
}

```

Ví dụ 2: Thêm phần tử vào cuối và xóa node tại vị trí

```

#include <iostream>
using namespace std;

```

```

struct Node {
    int data;
    Node* prev;
    Node* next;
    Node(int value) {
        data = value;
        prev = nullptr;
        next = nullptr;
    }
};

```

```

void insertAtTail(Node*& head, int value) {

```

```

Node* newNode = new Node(value);
if (head == nullptr) {
    head = newNode;
    return;
}
Node* temp = head;
while (temp->next != nullptr)
    temp = temp->next;
temp->next = newNode;
newNode->prev = temp;
}

void deleteAtPosition(Node*& head, int position) {
    if (head == nullptr || position < 1) return;
    Node* temp = head;
    int count = 1;
    if (position == 1) {
        head = head->next;
        if (head) head->prev = nullptr;
        delete temp;
        return;
    }
    while (temp != nullptr && count < position) {
        temp = temp->next;
        count++;
    }
    if (temp == nullptr) return;
    if (temp->prev) temp->prev->next = temp->next;
    if (temp->next) temp->next->prev = temp->prev;
    delete temp;
}

void display(Node* head) {
    cout << "Danh sách: ";
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    Node* head = nullptr;
    insertAtTail(head, 10);

```

```

insertAtTail(head, 20);
insertAtTail(head, 30);
insertAtTail(head, 40);
display(head);
deleteAtPosition(head, 2);
display(head);
return 0;
}

```

Ví dụ 3: Sử dụng Template để tạo danh sách liên kết đôi với kiểu dữ liệu bất kỳ

```

#include <iostream>
using namespace std;

template <typename T>
struct Node {
    T data;
    Node* prev;
    Node* next;

    Node(T value) {
        data = value;
        prev = nullptr;
        next = nullptr;
    }
};

template <typename T>
class DoublyLinkedList {
private:
    Node<T>* head;

public:
    DoublyLinkedList() {
        head = nullptr;
    }

    void insertAtTail(T value) {
        Node<T>* newNode = new Node<T>(value);
        if (head == nullptr) {
            head = newNode;
            return;
        }
        Node<T>* temp = head;
        while (temp->next != nullptr)

```

```

        temp = temp->next;
        temp->next = newNode;
        newNode->prev = temp;
    }

    void display() {
        Node<T>* temp = head;
        cout << "Danh sách: ";
        while (temp != nullptr) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
};

int main() {
    DoublyLinkedList<int> intList;
    intList.insertAtTail(1);
    intList.insertAtTail(2);
    intList.insertAtTail(3);
    intList.display();

    DoublyLinkedList<string> strList;
    strList.insertAtTail("one");
    strList.insertAtTail("two");
    strList.insertAtTail("three");
    strList.display();

    return 0;
}

```

Ví dụ 4: Danh sách liên kết đôi với cả con trỏ head và tail

```

#include <iostream>
using namespace std;

template <typename T>
struct Node {
    T data;
    Node* prev;
    Node* next;
    Node(T value) {
        data = value;
        prev = nullptr;
    }
};

```

```

        next = nullptr;
    }
};

template <typename T>
class DoublyLinkedList {
private:
    Node<T>* head;
    Node<T>* tail;

public:
    DoublyLinkedList() {
        head = tail = nullptr;
    }

    void insertAtTail(T value) {
        Node<T>* newNode = new Node<T>(value);
        if (!head) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }

    void insertAtHead(T value) {
        Node<T>* newNode = new Node<T>(value);
        if (!head) {
            head = tail = newNode;
        } else {
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
        }
    }

    void displayForward() {
        Node<T>* temp = head;
        cout << "Danh sách (từ đầu đến cuối): ";
        while (temp) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
};

```

```

    }

    void displayBackward() {
        Node<T>* temp = tail;
        cout << "Danh sách (từ cuối về đầu): ";
        while (temp) {
            cout << temp->data << " ";
            temp = temp->prev;
        }
        cout << endl;
    }
};

int main() {
    DoublyLinkedList<int> list;
    list.insertAtTail(10);
    list.insertAtTail(20);
    list.insertAtHead(5);
    list.insertAtTail(30);
    list.displayForward();
    list.displayBackward();
    return 0;
}

```

3. NỘI DUNG BÀI THỰC HÀNH

Bài 1: Menu quản lý danh sách số nguyên

- 1. Thêm phần tử vào đầu danh sách
- 2. Thêm phần tử vào cuối danh sách
- 3. Hiển thị danh sách từ đầu đến cuối
- 4. Hiển thị danh sách từ cuối về đầu
- 5. Xóa phần tử tại vị trí bất kỳ
- 6. Thoát

Bài 2: Ứng dụng – Quản lý danh sách sinh viên

- Thêm sinh viên vào cuối danh sách
- Hiển thị danh sách sinh viên
- Tìm sinh viên theo MSSV
- Xóa sinh viên theo MSSV

- Tìm sinh viên có điểm trung bình cao nhất

Bài 3: Đảo ngược danh sách liên kết đôi

- Viết hàm reverseList(Node*& head) đảo ngược thứ tự các phần tử trong danh sách.

Bài 4 (Tự chọn): Sắp xếp danh sách tăng dần theo giá trị

- Cài đặt thuật toán sắp xếp danh sách tăng dần bằng Bubble Sort hoặc Selection Sort.